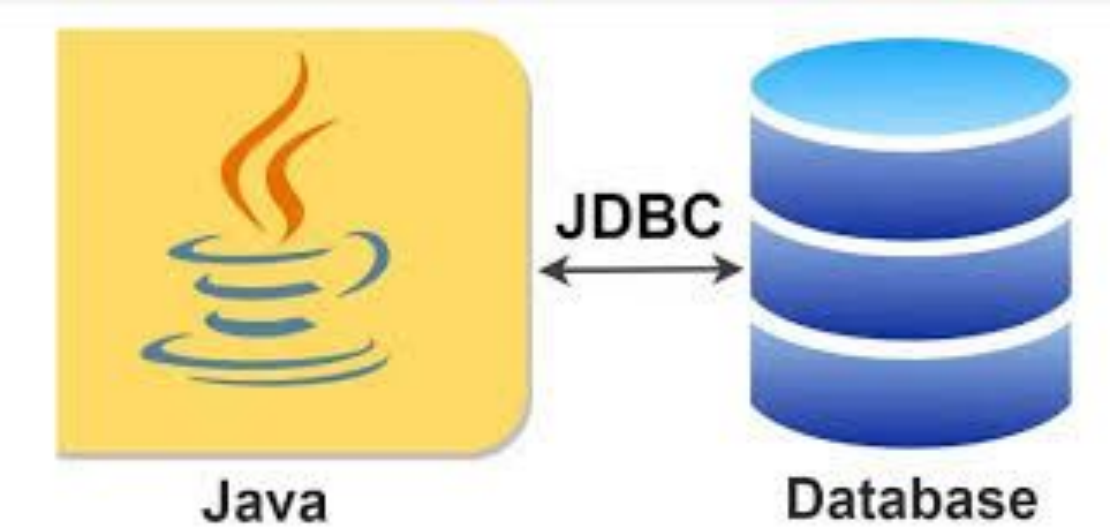
JDBC

JDBC - Java DataBase Connectivity



JDBC stands for **Java Database Connectivity**. It is a **Java API** that enables Java applications to interact with databases by connecting, querying, and updating data.

Key Features of JDBC:

- **API from Sun Microsystems** (now Oracle): JDBC is a specification that standardizes how Java applications connect and interact with databases.

- **Components**: It consists of **classes, interfaces, and exceptions** that help Java developers manage database connections and operations.

- **Technology, Not a Language**: JDBC is **not a programming language**; it's a **technology** or set of concepts used to develop database-driven Java applications.

Purpose of JDBC:

- To **write Java programs** that can **access and manipulate data** stored in relational databases (RDBs).
- To provide a **standard protocol** for Java applications to interact with **various databases and spreadsheets** using JDBC drivers.

Where JDBC is Used:

- **J2SE (Java Standard Edition)**: Core JDBC functionality.
- **J2EE (Java Enterprise Edition)**: Used along with **Servlets** and **JSP** for enterprise applications.
- **J2ME (Java Micro Edition)**: Used in **mobile applications**, although less common today due to modern frameworks.

1. J2SE (Java Standard Edition)

- This is the **basic version of Java** used to build general-purpose desktop or console applications.
- JDBC is included here to allow simple **database access in standalone Java programs**.

Example: A desktop app that stores user data in a database.

2. J2EE (Java Enterprise Edition)

- This is used for **building web-based or large-scale enterprise applications**.
- JDBC works **with Servlets and JSP (Java Server Pages)** to access databases in **web applications**.

Example: A company website where users log in and view their account information stored in a database.

3. J2ME (Java Micro Edition)

- This was used to build **Java applications for mobile devices** or embedded systems.
- JDBC was used here in a **limited form** to access local or lightweight databases.

Example: A mobile phone app (older generation) that stores contact info using a small database.

JDBC (Java Database Connectivity) defines a **set of rules (API)** provided by **Java vendors** (originally **Sun Microsystems**, now **Oracle**) that allow Java applications to interact with databases.

However, these rules need to be **implemented by database vendors** (like Oracle, MySQL, SQL Server, etc.). To do this, **driver software** is provided by each database vendor. This software follows the JDBC rules and allows your Java program to communicate with their specific database.

Why Do We Need JDBC Drivers?

- JDBC uses **SQL statements** like INSERT, SELECT, UPDATE, DELETE.
- These SQL statements behave slightly differently depending on the **database vendor**.
- So, each **database vendor (Oracle, MySQL, etc.)** provides a **JDBC driver** that translates JDBC calls into their specific database commands.

How it works

1. Java application sends JDBC commands.
2. JDBC driver converts them into database-specific commands.
3. Database processes the request.
4. Driver returns the result back to the Java application.

Features of JDBC:

1. **Standard API-** It provides a consistent way to access databases in Java.
2. **Database Independent-** You can switch databases (e.g., from MySQL to PostgreSQL) with minimal changes in code.
3. **Platform Independent-** Being part of Java, JDBC works across all platforms that support Java (Windows, Linux, Mac, etc.).
4. **Supports CRUD Operations -JDBC** allows basic operations, often called **CRUD** or **SCUD**:

    - **C** → Create (INSERT)
    - **R/S** → Retrieve/Select (SELECT)
    - **U** → Update (UPDATE)
    - **D** → Delete (DELETE)

5. **Advanced Features:** JDBC also supports:

- **Cursors** (to navigate through rows of data)
- **Views** (virtual tables)
- **Triggers** (automatic actions in the database)

Why Do We Need JDBC?

1. **To Connect Java with Databases-** JDBC provides a way to **connect a Java application** to any **relational database**, such as Oracle, MySQL, PostgreSQL, etc.

2. **To Execute SQL Queries-** With JDBC, we can run **SQL queries** like: SELECT (read data), INSERT (add data), UPDATE (modify data), DELETE (remove data)

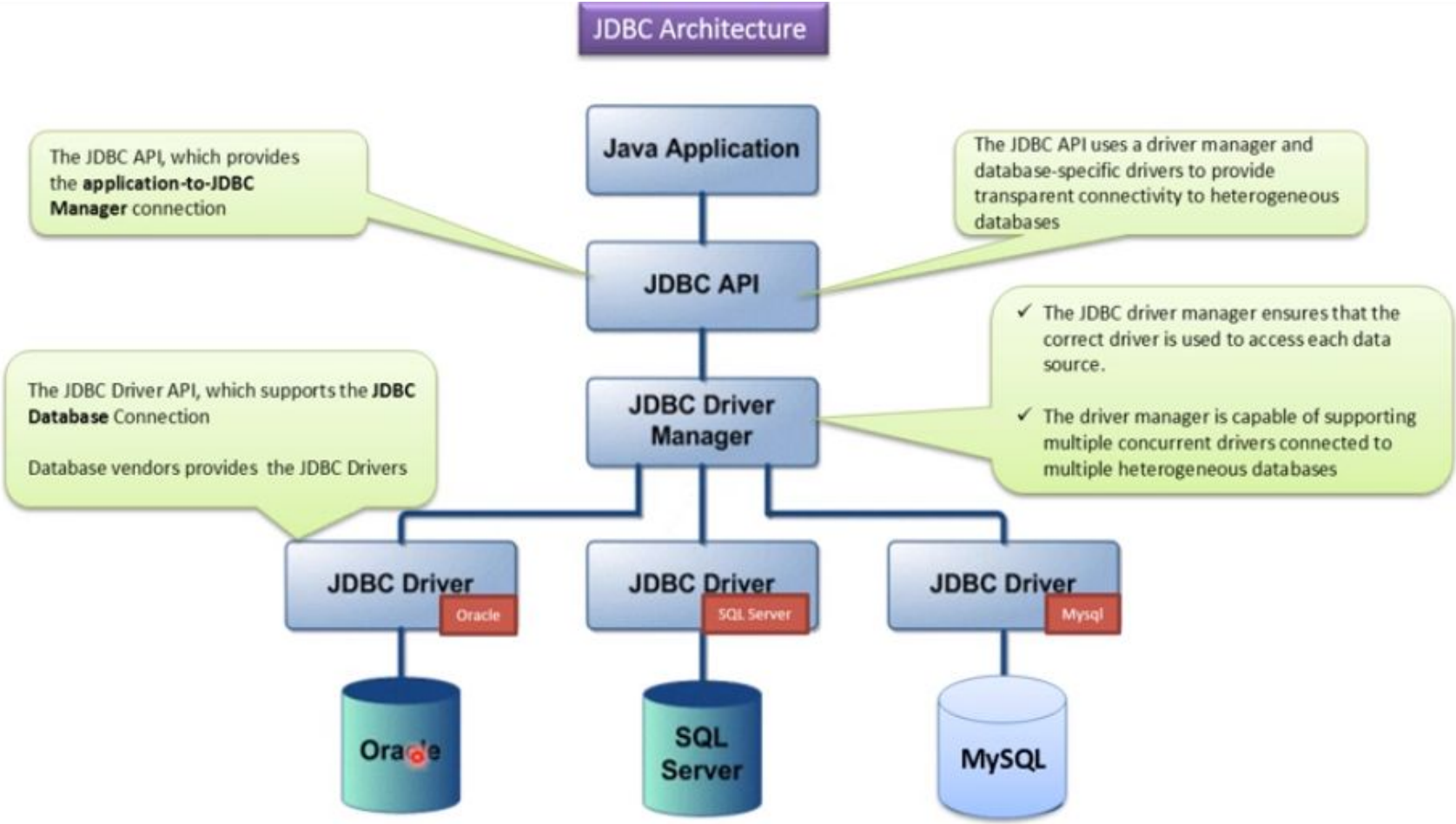
3. **To Work Across Platforms and Databases -** JDBC works with both:

    - **Java SE (Standard Edition)** – for desktop or console-based apps.

    - **Java EE (Enterprise Edition)** – for large-scale, web-based applications.

4. **To Support Multiple Databases with One API**
   No matter which database you're using, **the JDBC API stays the same**. You just need the correct **JDBC driver** for that specific database.

JDBC Architecture



The JDBC API supports both two-tier and three-tier processing models for database access but in general, JDBC Architecture consists of two layers −

- **JDBC API** − This provides the application-to-JDBC Manager connection.
- **JDBC Driver API** − This supports the JDBC Manager-to-Driver Connection.

The JDBC API uses a driver manager and database-specific drivers to provide transparent connectivity to heterogeneous databases.

The JDBC driver manager ensures that the correct driver is used to access each data source. The driver manager is capable of supporting multiple concurrent drivers connected to multiple heterogeneous databases.

Drivers, which are used to connect Java applications with databases. **Due to the variety of operating systems and hardware on which Java runs,** Sun Microsystems (now Oracle) defined **four types of JDBC drivers** to suit different environments:

Types of JDBC drivers

JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand. There are 4 types of JDBC drivers:

1. **Type-1 driver or JDBC-ODBC bridge driver**
2. **Type-2 driver or Native-API driver (partially java driver)**
3. **Type-3 driver or Network Protocol driver (fully java driver)**
4. **Type-4 driver or Thin driver (fully java driver)**

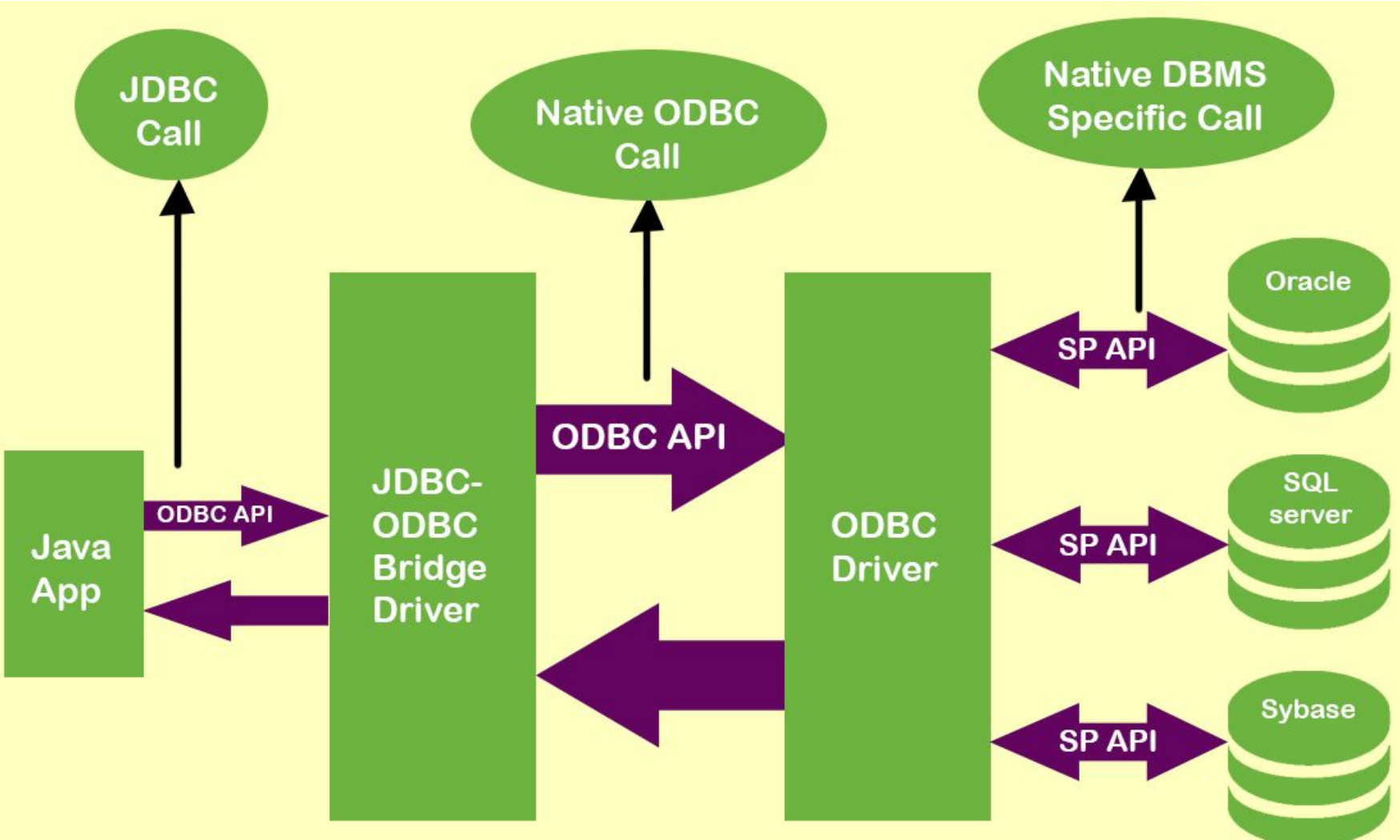
1. Type-1 driver or JDBC-ODBC bridge driver

- This driver **acts as a bridge** between **JDBC** and **ODBC**.
- JDBC calls from the Java program are translated into ODBC function calls.
- These ODBC calls are then used to interact with the actual database.
- You need to have an **ODBC driver** for the target database installed on the client machine.
- A **DSN (Data Source Name)** must be configured in the system ODBC settings to point to the database.
- **sun.jdbc.odbc.JdbcOdbcDriver** class is **no longer included in Java 8+**.

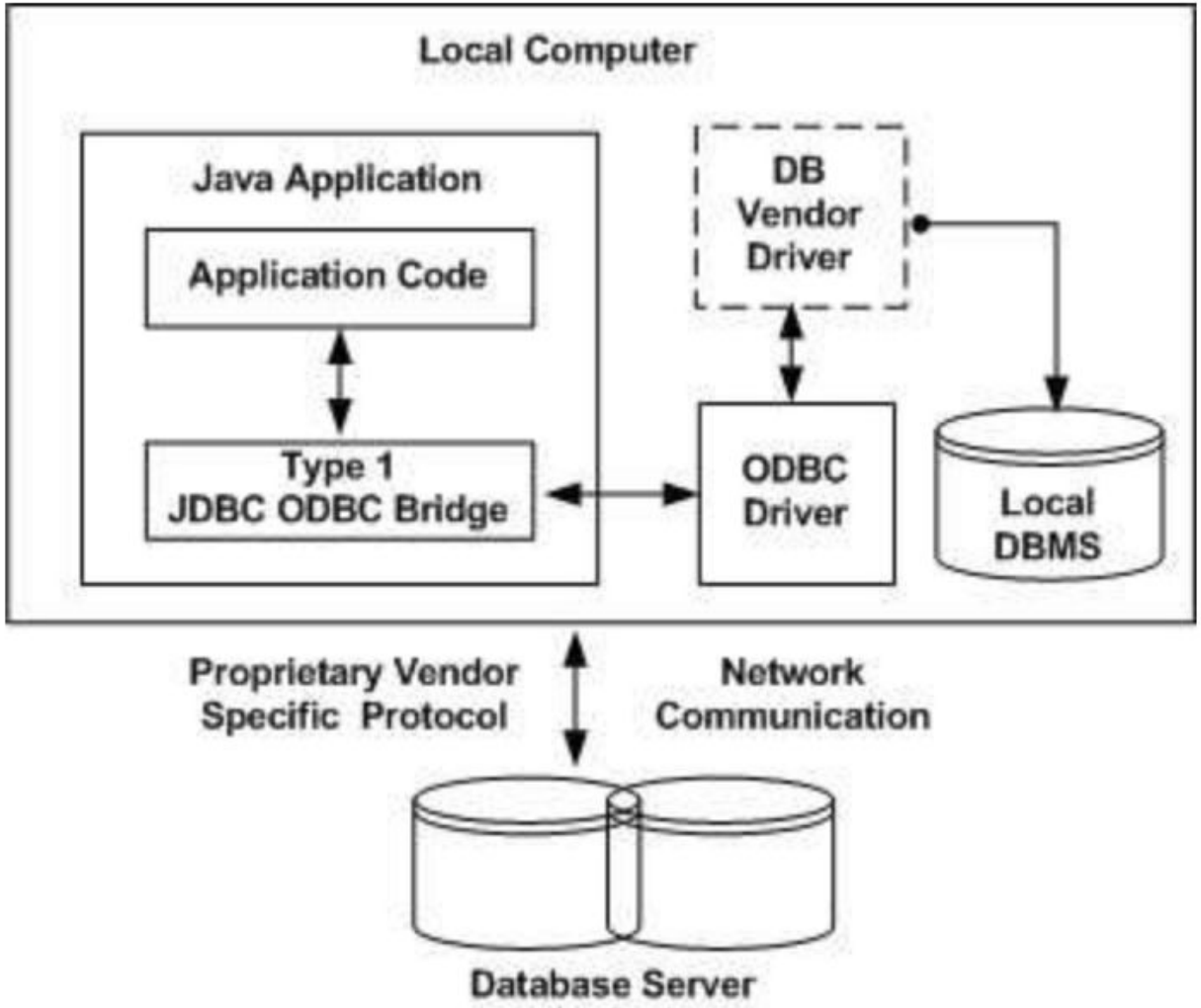
Java → JDBC → ODBC → Database

A **DSN (Data Source Name)** must be set up on your computer. This tells ODBC how to find your database.

An **ODBC driver** must be installed for the database (like MS Access, SQL Server, etc.).


JDBC Call
Native ODBC Call
Native DBMS Specific Call
Java App
ODBC API
JDBC-ODBC Bridge Driver
ODBC API
ODBC Driver
SP API
Oracle
SP API
SQL server
SP API
Sybase

1. The Java application makes the JDBC call to the JDBC-ODBC bridge driver to access a data source.
2. The JDBC-ODBC bridge driver resolves the JDBC call and makes an equivalent ODBC call to the ODBC driver.
3. The ODBC driver completes the request and sends responses to the JDBC-ODBC bridge driver.
4. The JDBC-ODBC bridge driver converts the response into JDBC standards and displays the result to the requesting Java application.


Local Computer
Java Application
Application Code
Type 1
JDBC ODBC Bridge
DB
Vendor
Driver
ODBC
Driver
Local
DBMS
Proprietary Vendor
Specific Protocol
Network
Communication
Database Server

Advantages of Type 1 JDBC Driver:

1. **Quick to Set Up (for Early Development):**Easy to use if a **DSN is already configured** and helpful when you're just **testing or prototyping** Java programs with databases.

2. **Access to Any ODBC-Supported Database:** Can connect to **many types of databases** (like MS Access, SQL Server, Oracle) as long as there's an ODBC driver for it.

3. **No Need for Native Database Libraries in Java:** Java doesn’t need to directly support the database — **ODBC handles that** part.

4. **Good for Learning Purposes:** Great for students or beginners who want to try connecting Java to a database **without installing extra JDBC drivers**.

Disadvantages:

- Platform dependent (requires native ODBC drivers).
- Performance is relatively poor due to multiple layers (JDBC → ODBC → DB).
- Not suitable for web-based applications.
- **Deprecated in Java 8 and later.**

Use Case:

- Experimental or legacy applications.
- When no pure Java driver is available.

Summary

1. **JDBC-ODBC Bridge Driver (Type 1):**
    - Connects Java applications to databases via ODBC (Open Database Connectivity).
    - Requires ODBC drivers to be installed on the client machine.
    - Platform dependent and generally considered outdated.

Type-2 driver or Native-API driver (partially java driver)

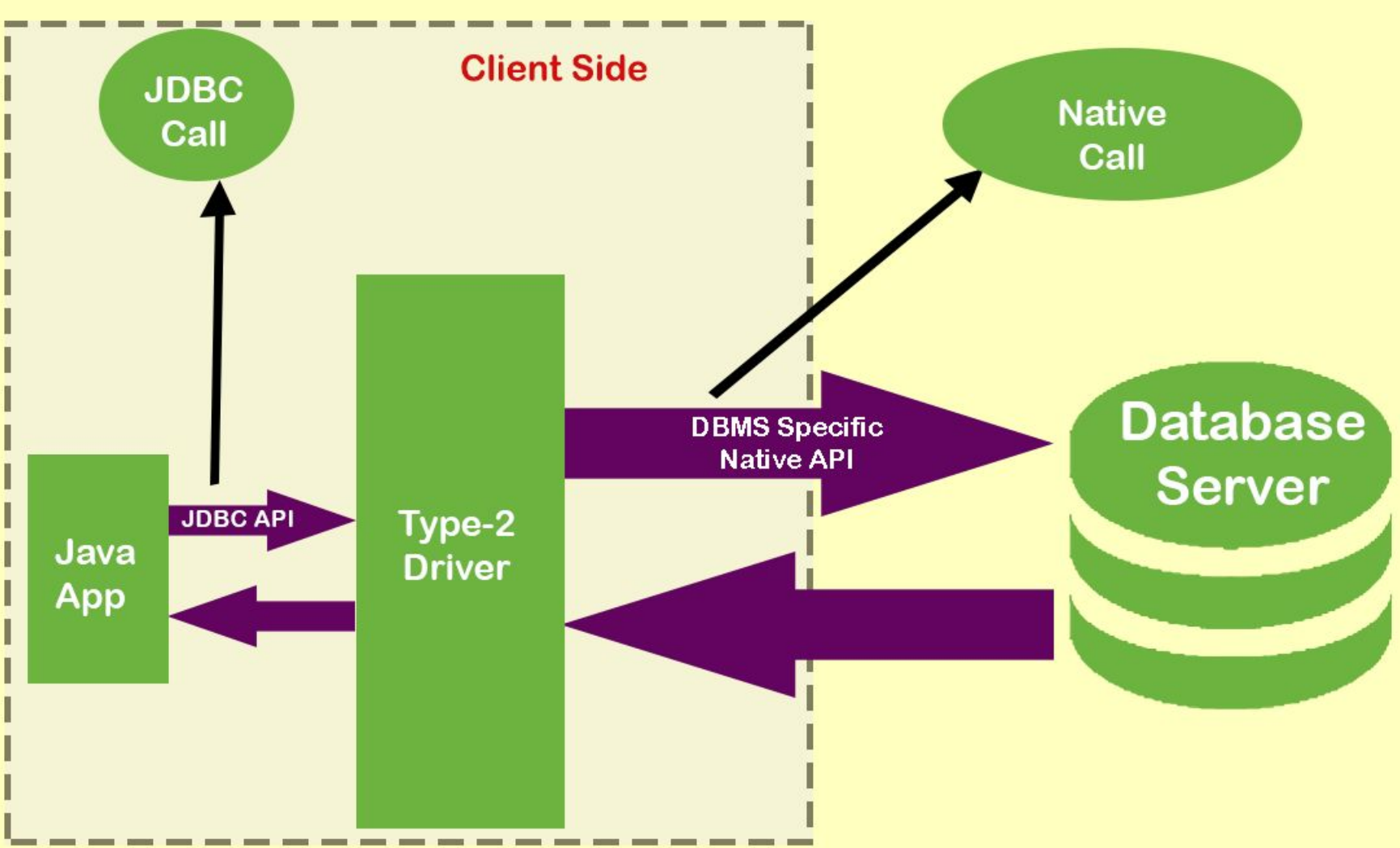
- In a Type 2 driver, JDBC API calls are converted into native C/C++ API calls, which are unique to the database.
- These drivers are typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge.
- The vendor-specific driver must be installed on each client machine.
- If we change the Database, we have to change the native API, as it is specific to a database and they are mostly obsolete now, but you may realize some speed increase with a Type 2 driver, because it eliminates ODBC's overhead.

Java → JDBC → Native C/C++ API → Database

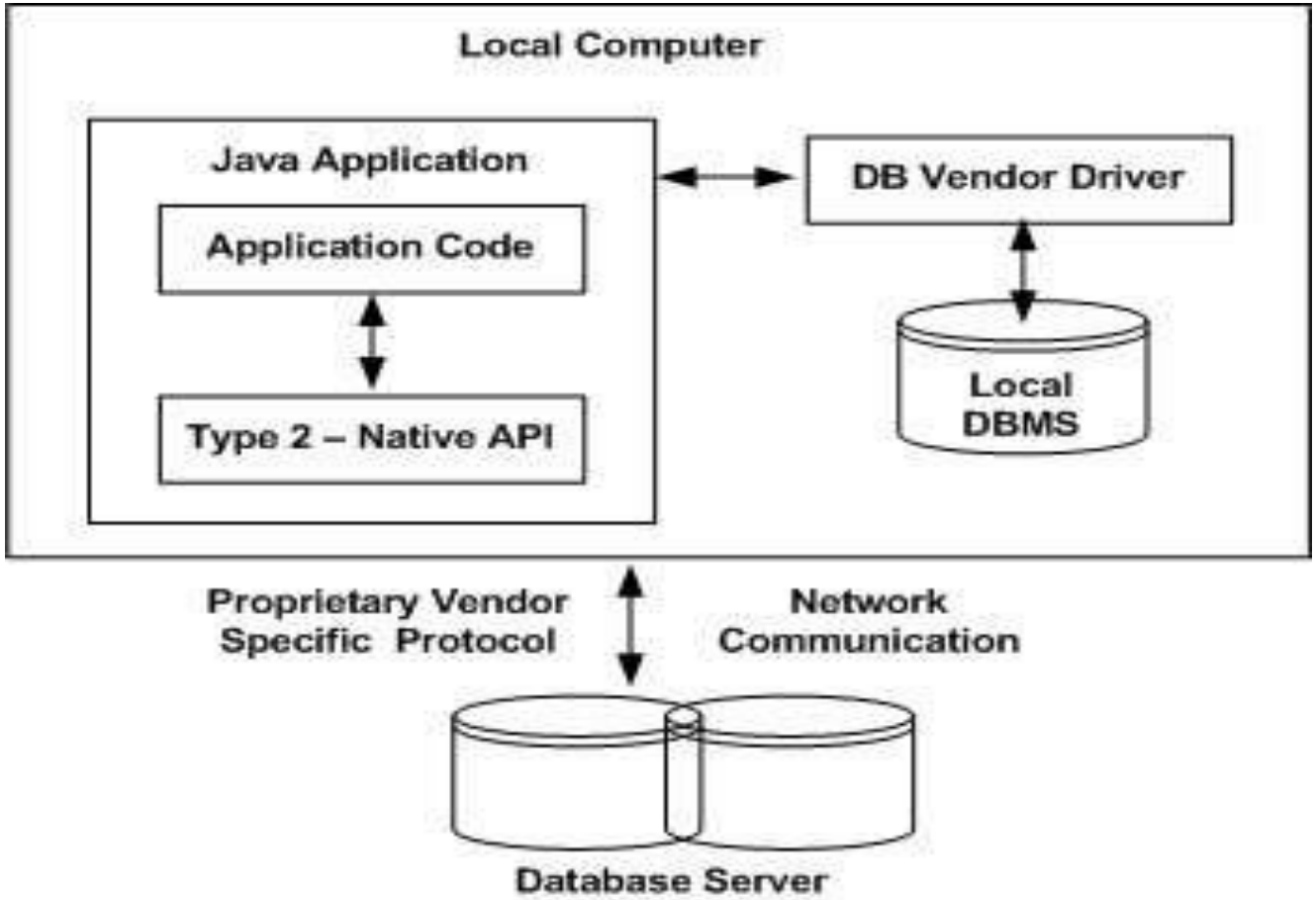
So, your Java code sends commands to JDBC, which then passes them to the **database's native library**, which talks to the database.

Advantages:

- Faster than Type 1 -No ODBC layer means less overhead.
- Better performance -Uses optimized native libraries from the database vendor.
- Vendor support-Often provided and supported directly by the database maker-**Drivers are often developed and maintained directly by the database company.**


Client Side
JDBC Call
Native Call
Java App
JDBC API
Type-2 Driver
DBMS Specific Native API
Database Server

- As shown above figure, the Java application that needs to communicate with the database is programmed using the JDBC API.
- These JDBC calls (programs written by using JDBC API) are converted into database-specific native calls in the client machine and therefore the request is then dispatched to the database-specific native libraries.
- These native libraries present in the client are intelligent enough to send the request to the database server by using a native protocol.


Local Computer
Java Application
Application Code
Type 2 – Native API
DB Vendor Driver
Local
DBMS
Proprietary Vendor
Specific Protocol
Network
Communication
Database Server

Disadvantages:

- **Platform dependent** (because of native code).
- Not portable — you have to **change drivers if you change the database**.
- Requires **extra installation/configuration** on every client.
- Mostly **obsolete** now, replaced by Type 4 drivers.

Summary:

2. Native-API Driver (Type 2):

- Converts JDBC calls into database-specific native calls using native libraries.
- Requires database client libraries to be installed.
- Offers better performance than Type 1 but is still platform dependent.

Type-3 driver or Network Protocol driver (fully java driver)

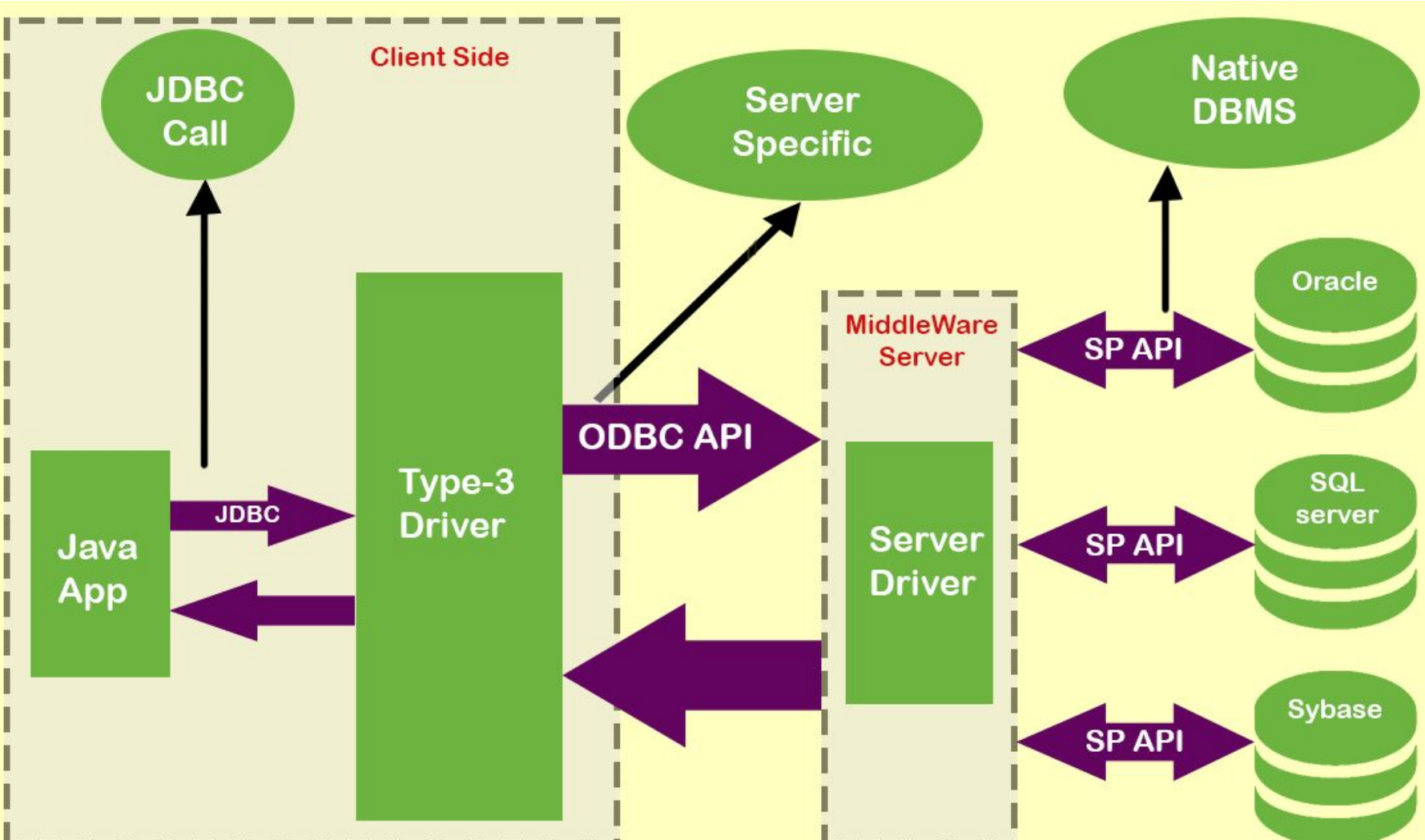
The **Type 3 driver** uses a **middleware server** between your Java application and the database.

It works in **two steps**:

1. Your Java app makes a JDBC call.
2. The driver sends that call to a middleware server.
3. The middleware server translates the call into a database-specific call and sends it to the actual database.

Java → JDBC → **Middleware Server** → Database

- Fully written in **Java** (platform independent).
- Connects to the database over a **network (socket)**.
- The **middleware server handles the database-specific logic**.
- Also called **Net Protocol Driver** or **Middleware Driver**.


Client Side
JDBC Call
Server Specific
Native DBMS
Java App
JDBC
Type-3 Driver
ODBC API
MiddleWare Server
Server Driver
SP API
Oracle
SP API
SQL server
SP API
Sybase

Advantages of Type 3 Driver:

1. **Platform Independent-** Written entirely in Java – works on any operating system.

2. **Supports Multiple Databases-** The middleware server can communicate with different databases without changing the Java code.

3. **Middleware Features-** Middleware can handle **connection pooling, load balancing, security**, and **performance optimizations**.

4. **Good for Web-Based Applications and Applets-** The driver can be downloaded automatically with the application (ideal for applets).

5. **No Need for Native Libraries-** Unlike Type 1 or 2, there's **no need to install native database libraries** on the client machine.

Disadvantages of Type 3 Driver:

1. **Requires a Middleware Server-** You must install and maintain a separate middleware server.

2. **More Complex Setup-** Setting up and configuring the middleware adds complexity.

3. **Network Overhead-** Slightly slower than Type 4 due to an extra network hop (Java → middleware → database).

4. **Vendor Dependent Middleware-** Middleware is usually vendor-specific and may have licensing or compatibility issues.

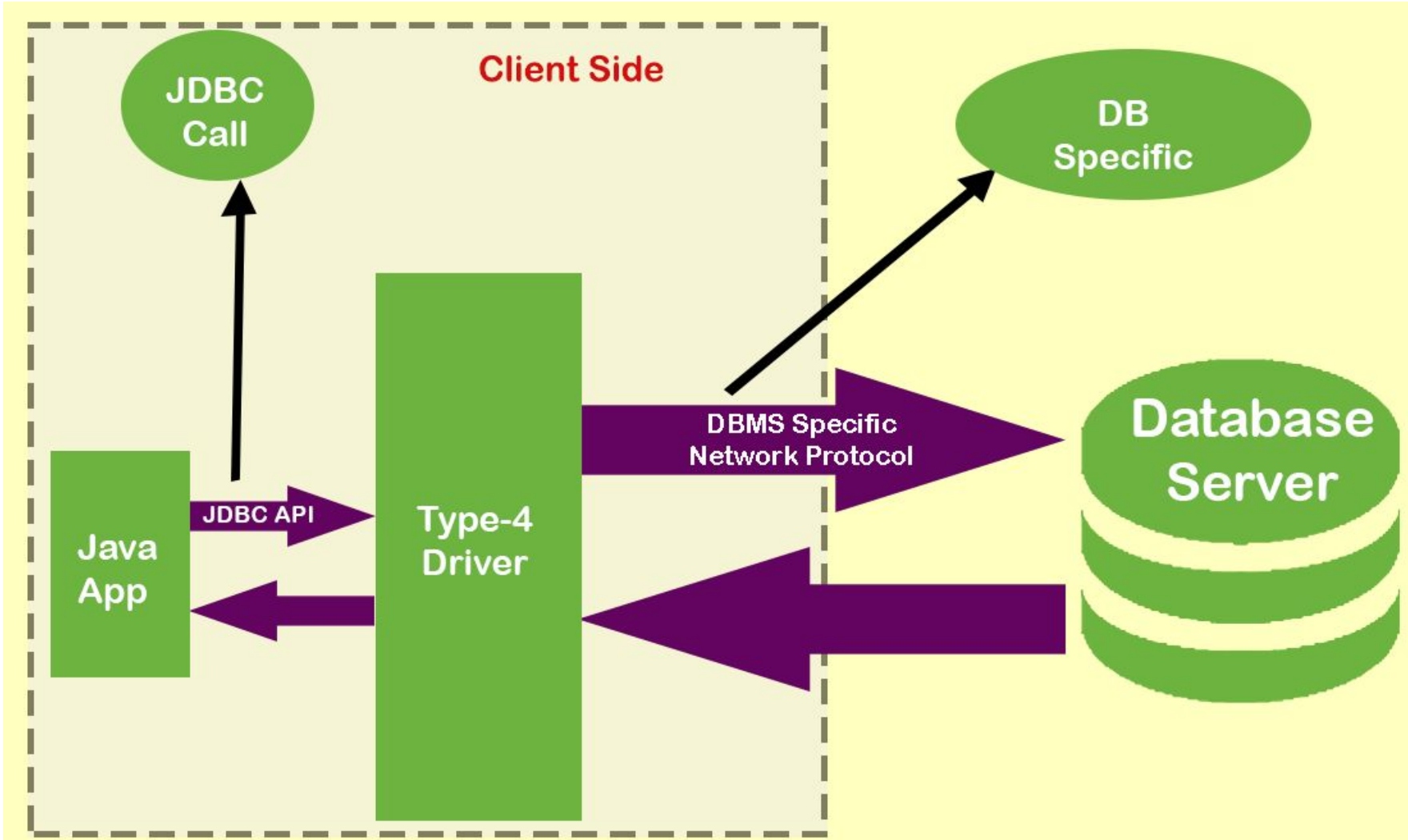
Summary:

3. Network-Protocol Driver (Middleware Driver) (Type 3):

- Translates JDBC calls into a database-independent network protocol, which a middleware server then converts to DB-specific protocol.
- Suitable for internet-based applications.
- More flexible and platform-independent.

Type-4 driver or Thin driver (fully java driver)

- The **Type 4 driver** is a **pure Java driver** that directly converts JDBC calls into **DBMS-specific network messages**.
- It **connects directly to the database** server using the **proprietary protocol** of the database.
- It's also known as a **"thin driver"** because it's lightweight and doesn't require any extra software or native code.
- **Java App → Type 4 JDBC Driver → Database Server (via socket)**
- The driver **prepares DBMS-specific messages** and sends them over a **network socket**.
- Typically **implemented by database vendors** (e.g., Oracle, MySQL).


Client Side
JDBC Call
DB Specific
DBMS Specific Network Protocol
Database Server
JDBC API
Java App
Type-4 Driver

Advantages of Type 4 Driver:

1. P**ure Java – Platform Independent-** Works on any system with a JVM (no native code).

2. **High Performance-** Direct connection to the database — **no ODBC, no middleware, no extra layers**.

3. **Lightweight ("Thin")-** Easy to use, small size, and fast to load.

4. **No Additional Software Needed-** No need to install any libraries or native drivers on the client machine.

5. **Auto Downloadable for Client-Side Applications-** Can be included in applications and **downloaded automatically** (great for applets or Java Web Start apps).

6. **Good for Server-Side Applications-** Ideal for **web servers, enterprise applications**, and cloud-based systems.

7. **Officially Supported by DB Vendors-** Developed and maintained by **database vendors**, ensuring better reliability and updates.

Disadvantages of Type 4 Driver:

1. **Database-Specific**
    - Each database has its **own Type 4 driver** (e.g., Oracle driver won't work with MySQL).
2. **Driver Maintenance**
    - If the **database protocol changes**, the driver must be updated accordingly.

Summary

4. Database-Protocol Driver (Pure Java Driver) (Type 4):

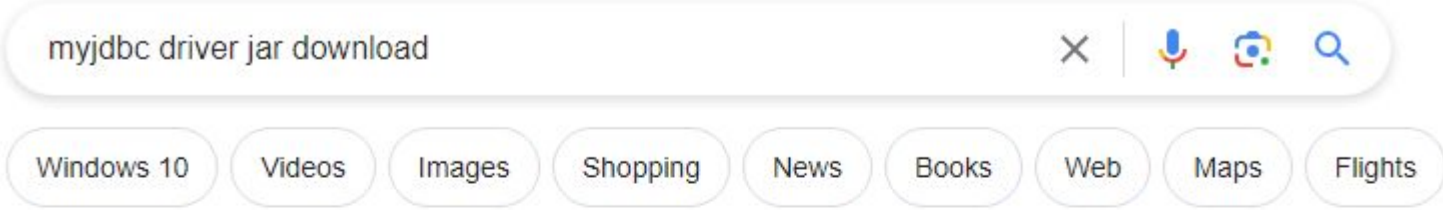
- Converts JDBC calls directly into the database-specific protocol.
- Written entirely in Java—no native libraries needed.
- Platform-independent and offers best performance.

Which Driver should be used

- If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is 4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations where a type 3 or type 4 driver is not available yet for your database.
- The type 1 driver is not considered a deployment-level driver and is typically used for development and testing purposes only.

Steps for connecting Database

1. Import package
2. Loading driver class & registers–Class.forName(“com.mysql.jdbc.Driver”);
3. Establishing the connection—> Connection con=DriverManager.getConnection(“ “);
4. Create statement—->Use statement interface→Statement st=con.createStatement();
5. Execute SQL query—>2 methods 1. executeUpdate()-->insert delete update (it returns integer value) 2.executeQuery()--->select
6. Process the result —> results will be stored in resultset
7. Close the connection


myjdbc driver jar download
Windows 10
Videos
Images
Shopping
News
Books
Web
Maps
Flights
Showing results for my jdbc driver jar download
Search instead for myjdbc driver jar download
MySQL :: Developer Zone
https://dev.mysql.com › downloads › connector
MySQL :: Download Connector/J
MySQL Connector/J is the official JDBC driver for MySQL. MySQL Connector/J 8.0 and higher is compatible with all MySQL versions starting with MySQL 5.7.

MySQL Community Downloads

Connector/J

General Availability (GA) Releases | Archives

Connector/J 8.4.0

Select Operating System:

Platform Independent

Platform Independent (Architecture Independent), Compressed TAR Archive (mysql-connector-j-8.4.0.tar.gz)	8.4.0	4.1M	Download
		MD5: 33a74d1a803b504a1141a327bf6ff7c3 | Signature	
Platform Independent (Architecture Independent), ZIP Archive (mysql-connector-j-8.4.0.zip)	8.4.0	4.9M	Download
		MD5: 5d426b639f7b4c314492edff07c615dc | Signature	

We suggest that you use the MD5 checksums and GnuPG signatures to verify the integrity of the packages you download.

MySQL Community Downloads

Login Now or Sign Up for a free account.

An Oracle Web Account provides you with the following advantages:

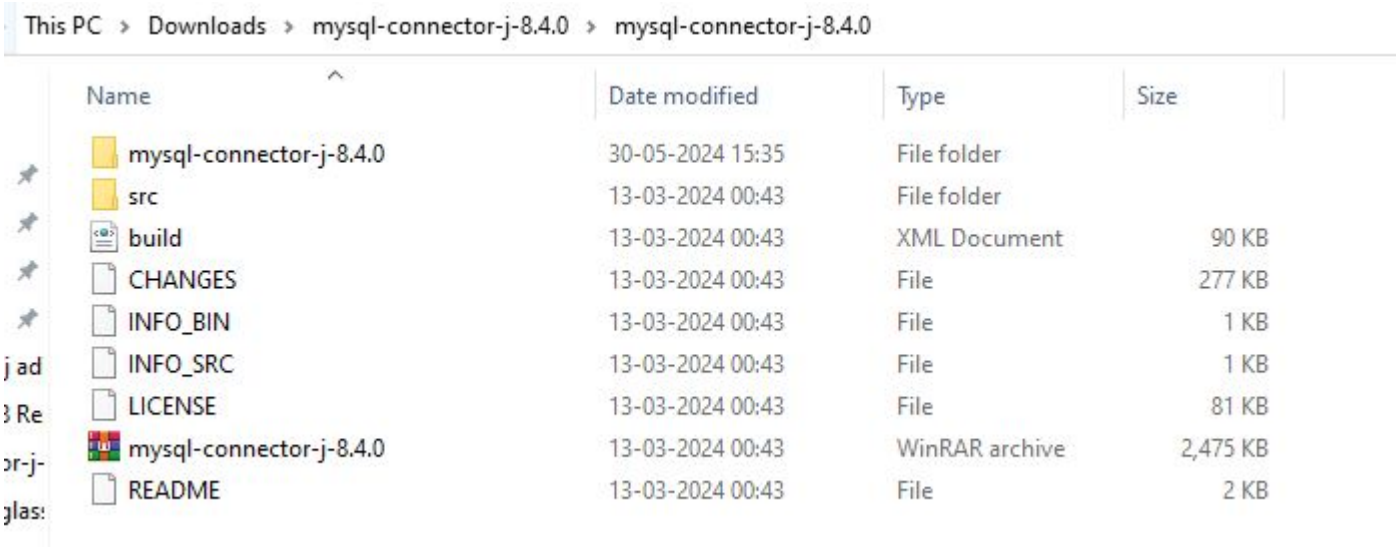
- Fast access to MySQL software downloads
- Download technical White Papers and Presentations
- Post messages in the MySQL Discussion Forums
- Report and track bugs in the MySQL bug system

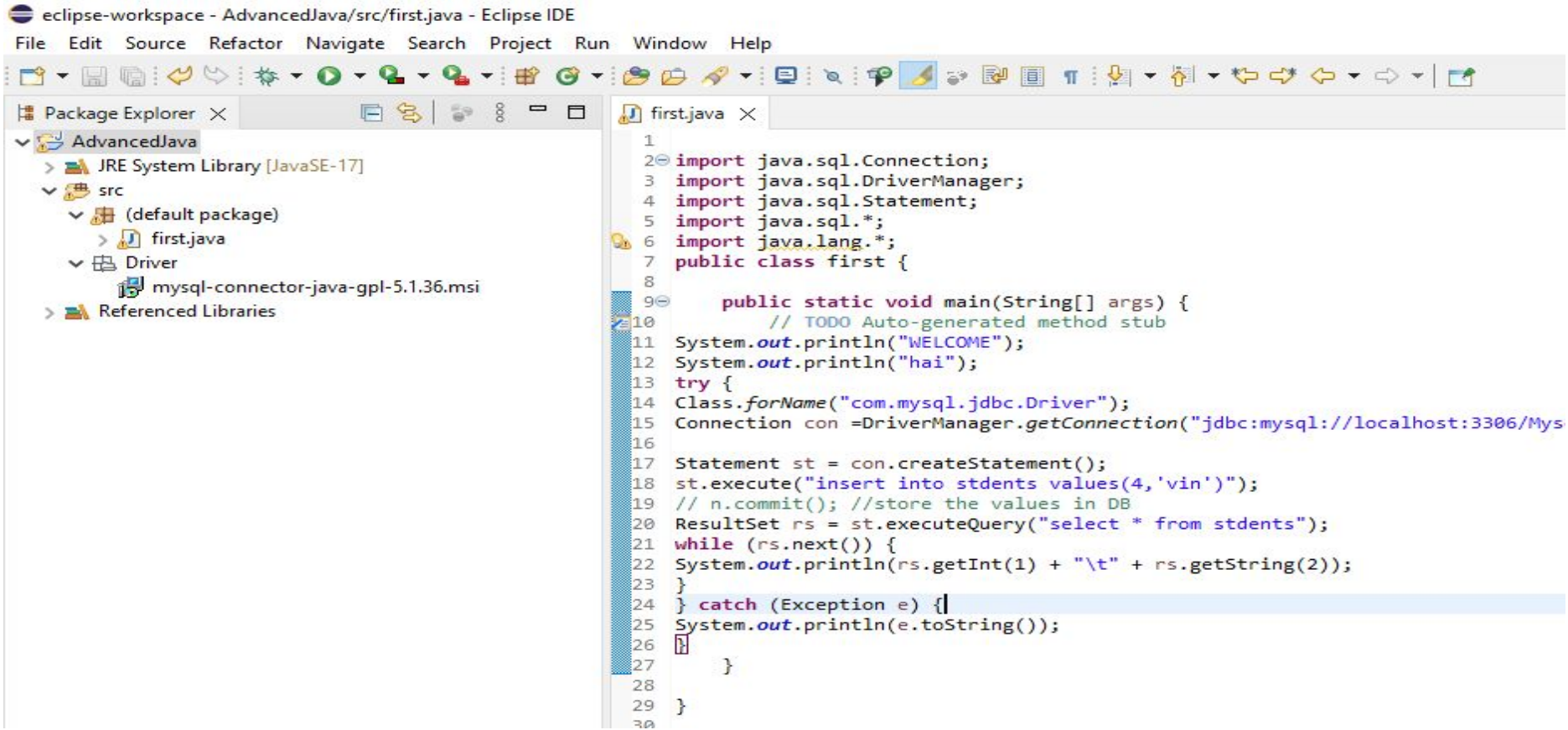
Login »
using my Oracle Web account

Sign Up »
for an Oracle Web account

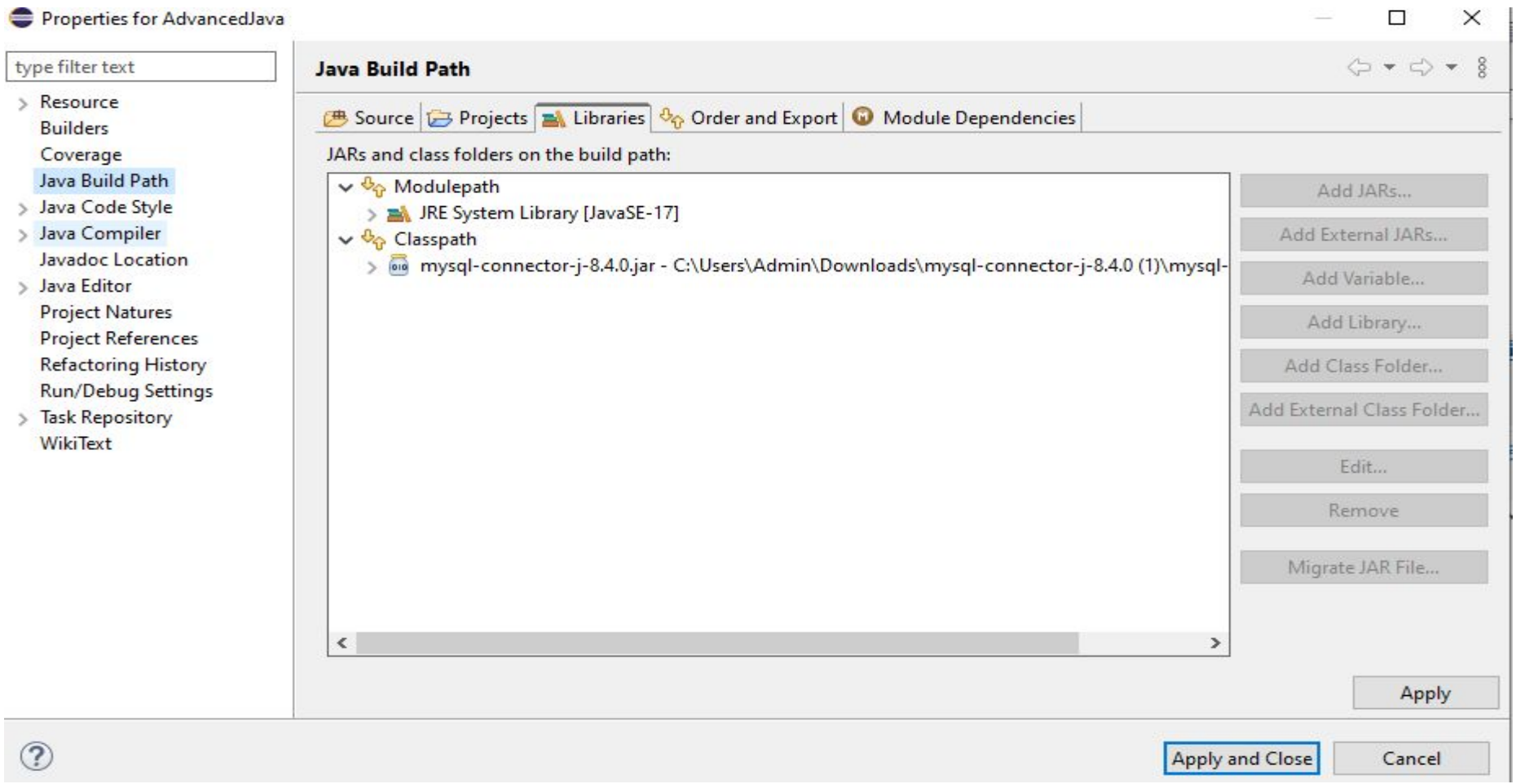
MySQL.com is using Oracle SSO for authentication. If you already have an Oracle Web account, click the Login link. Otherwise, you can signup for a free account by clicking the Sign Up link and following the instructions.

No thanks, just start my download.

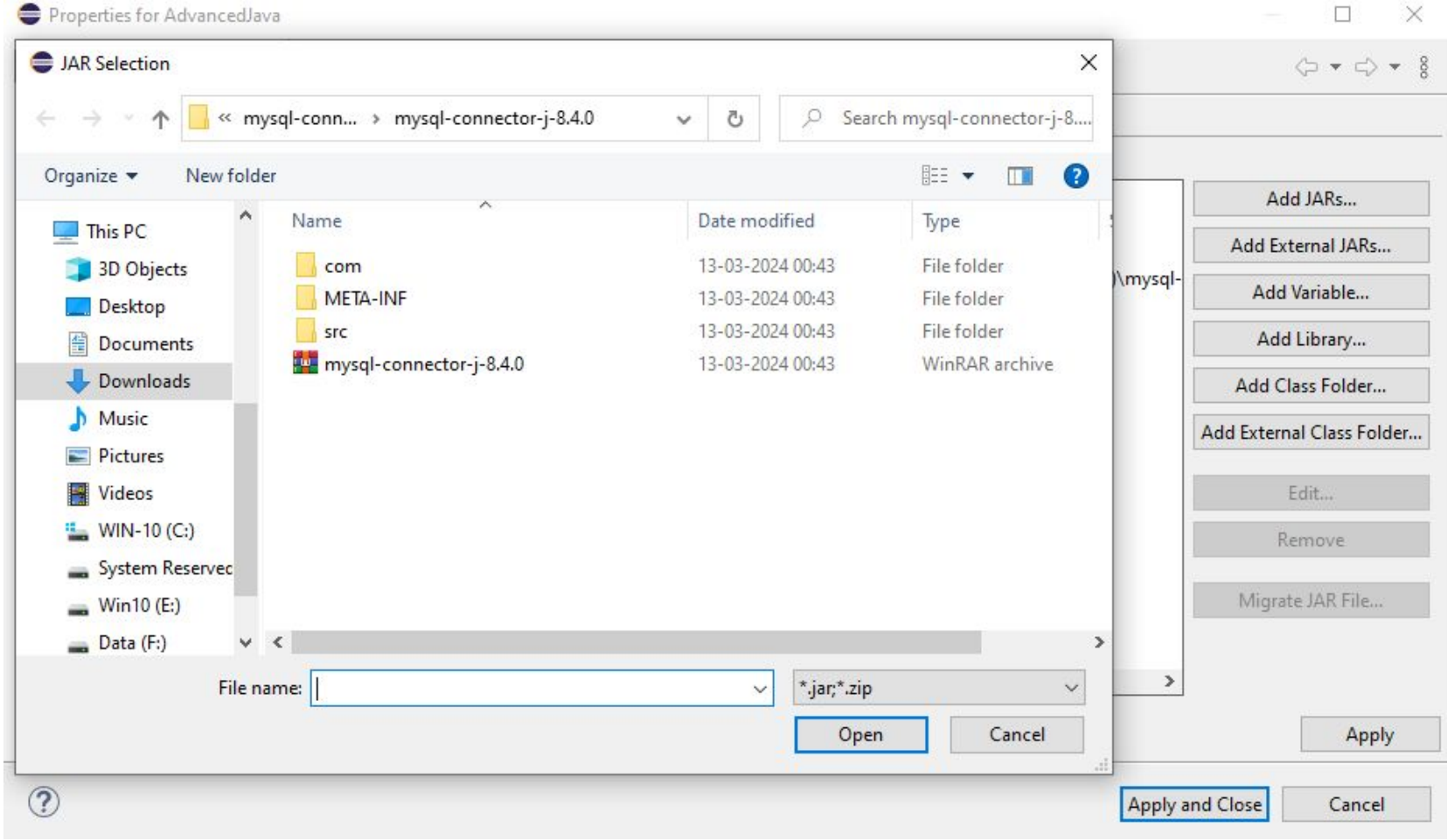

This PC › Downloads › mysql-connector-j-8.4.0 › mysql-connector-j-8.4.0
Name
Date modified
Type
Size
mysql-connector-j-8.4.0
30-05-2024 15:35
File folder
src
13-03-2024 00:43
File folder
build
13-03-2024 00:43
XML Document
90 KB
CHANGES
13-03-2024 00:43
File
277 KB
INFO_BIN
13-03-2024 00:43
File
1 KB
INFO_SRC
13-03-2024 00:43
File
1 KB
LICENSE
13-03-2024 00:43
File
81 KB
mysql-connector-j-8.4.0
13-03-2024 00:43
WinRAR archive
2,475 KB
README
13-03-2024 00:43
File
2 KB



Copy the jar file and paste it in folder(Driver) created in AdvancedJAva



Select properties—->java build path—>libraries



Select Add external JARS and open mysql jar file

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.*;
import java.lang.*;
public class first {
        public static void main(String[] args) {
                // TODO Auto-generated method stub
System.out.println("WELCOME");
System.out.println("hai");
try {
Class.forName("com.mysql.jdbc.Driver");
Connection con =DriverManager.getConnection("jdbc:mysql://localhost:3306/Mysqlcl", "root",
"password");
Statement st = con.createStatement();
st.execute("insert into stdents values(4,'vin')");
// n.commit(); //store the values in DB
ResultSet rs = st.executeQuery("select * from stdents");
while (rs.next()) {
System.out.println(rs.getInt(1) + "\t" + rs.getString(2));
}
} catch (Exception e) {
System.out.println(e.toString());
}
        }
}
```

OUTPUT:

```
WELCOME
hai
Loading class `com.mysql.jdbc.Driver'. This is deprecated. The new driver class is `com.mysql.cj.jdbc.Driver'. The driver is automatically registered via the SPI and manual loading of the driver class is generally unnecessary.
4       vin
6       sin
7       mit
```

simple Customers GUI example in Java using **Swing** and **JDBC** to:

1. Connect to a MySQL database
2. Display customer records in a table
3. Insert a new customer via a form

```
CREATE TABLE customers (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
```

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.*;
import java.sql.*;

public class CustomerGUI extends JFrame {
    // DB connection variables
    static final String DB_URL = "jdbc:mysql://localhost:3306/your_database";
    static final String USER = "root";
    static final String PASS = "your_password";
```

```
JTextField idField, nameField, emailField;
    JButton addButton;
    JTable table;
    DefaultTableModel tableModel;

    public CustomerGUI() {
    setTitle("Customer Management");
    setSize(600, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLayout(new BorderLayout());
```

```
JPanel formPanel = new JPanel(new GridLayout(4, 2));
        formPanel.add(new JLabel("ID:"));
        idField = new JTextField();
        formPanel.add(idField);

        formPanel.add(new JLabel("Name:"));
        nameField = new JTextField();
        formPanel.add(nameField);

        formPanel.add(new JLabel("Email:"));
        emailField = new JTextField();
        formPanel.add(emailField);
```

```
addButton = new JButton("Add Customer");
        formPanel.add(addButton);
        add(formPanel, BorderLayout.NORTH);
        // Table Panel
        tableModel = new DefaultTableModel(new String[]{"ID", "Name", "Email"}, 0);
        table = new JTable(tableModel);
        JScrollPane scrollPane = new JScrollPane(table);
        add(scrollPane, BorderLayout.CENTER);
        // Load data on startup
        loadCustomers();
```

```
// Add ActionListener
        addButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
                addCustomer();
        }
        });
        setVisible(true);
        }
        // Method to load data from DB to table
        private void loadCustomers() {
        try {
        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        Statement stmt = conn.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT * FROM customers");
        while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                String email = rs.getString("email");
                tableModel.addRow(new Object[]{id, name, email});
        }
        rs.close();
        stmt.close();
        conn.close();
        } catch (Exception ex) {
        ex.printStackTrace();
        }       }
```

```
// Method to insert new customer
    private void addCustomer() {
    try {
        int id = Integer.parseInt(idField.getText());
        String name = nameField.getText();
        String email = emailField.getText();
        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        PreparedStatement ps = conn.prepareStatement("INSERT INTO customers
(id, name, email) VALUES (?, ?, ?)");
        ps.setInt(1, id);
        ps.setString(2, name);
```

```
ps.setString(3, email);
            ps.executeUpdate();
            // Update table
            tableModel.addRow(new Object[]{id, name, email});
 JOptionPane.showMessageDialog(this, "Customer added successfully!");
            // Clear fields
            idField.setText("");
            nameField.setText("");
            emailField.setText("");
            ps.close();
            conn.close();
        }
```

```
catch (SQLIntegrityConstraintViolationException dup) {
            JOptionPane.showMessageDialog(this, "Customer ID already exists!", "Error",
JOptionPane.ERROR_MESSAGE);
    } catch (Exception ex) {
            ex.printStackTrace();
    }    }
    public static void main(String[] args) {
    try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            new CustomerGUI();
    } catch (Exception e) {
            e.printStackTrace();
    }       }    }
```

e.printStackTrace();

The call stack showing where the error occurred — line by line from the point of the exception up through the method call hierarchy

java.lang.ArithmeticException: / by zero

at MyClass.main(MyClass.java:3)